

# Containerization\_and\_DevOps

## Experiment 5:- Docker - Volumes, Environment Variables, Monitoring & Networks

### Part 1: Docker Volumes - Persistent Data Storage

#### Lab 1: Understanding Data Persistence

##### The Problem: Container Data is Ephemeral

```
docker run -it --name test-container ubuntu /bin/bash
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -it --name test-container ubuntu /bin/bash
root@9ed18ed4baab:/# echo "Hello World" > /data/message.txt
```

```
echo "Hello World" > /data/message.txt
cat /data/message.txt
exit
```

```
root@9ed18ed4baab:/# mkdir /data
echo "Hello World" > /data/message.txt
exit
exit
```

```
docker start test-container
docker exec test-container cat /data/message.txt
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker start test-container
docker exec test-container cat /data/message.txt
test-container
Hello World
```

##### Solution: Docker Volumes

## Lab 2: Volume Types

### Anonymous Volume

```
docker run -d -v /app/data --name web1 nginx
docker volume ls
docker inspect web1 | grep -A 5 Mounts
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d -v /app/data --name web1 nginx
docker volume ls
docker inspect web1 | grep -A 5 Mounts
12756ef2028d9036838c76454b9b7abd4522ceb121207bd6f43a2db1425b77d2
DRIVER      VOLUME NAME
local      assignment1_pgdata
local      caaec63d137f1153e400de0ced5c7fee4dd3ac58e966e75fa095a9a07720630b
local      containerassgn_pgdata
local      containerzatiassgn_postgres_data
local      dockerassignment_postgres_data
local      minikube
local      mydata
local      myvolume
local      myvolumej
local      project_pg_data
local      projectassignment_pgdata
"Mounts": [
  {
    "Type": "volume",
    "Name": "caaec63d137f1153e400de0ced5c7fee4dd3ac58e966e75fa095a9a07720630b",
    "Source": "/var/lib/docker/volumes/caaec63d137f1153e400de0ced5c7fee4dd3ac58e966e75fa095a9a07720630b/_data",
    "Destination": "/app/data",
    "Driver": "local",
    "Mode": "rw"
  }
]
```

### Named Volume

```
docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
docker volume ls
docker volume inspect mydata
```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume create mydata
docker run -d -v mydata:/app/data --name web2 nginx
docker volume ls
docker volume inspect mydata
mydata
9bbc1e429fee92065e1a89bb01ebed6ad5231ef4b8ad5ac1dcc15db4f2e63729
DRIVER      VOLUME NAME
local       assignment1_pgdata
local       caaec63d137f1153e400de0ced5c7fee4dd3ac58e966e75fa095a9a07720630b
local       containerassgn_pgdata
local       containerzatiassgn_postgres_data
local       dockerassignment_postgres_data
local       minikube
local       mydata
local       myvolume
local       myvolumej
local       project_pg_data
local       projectassignment_pgdata
[
  {
    "CreatedAt": "2026-03-20T21:05:30Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
    "Name": "mydata",
    "Options": null,
    "Scope": "local"
  }
]
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |

```

## Bind Mount

```

mkdir ~/myapp-data
docker run -d -v ~/myapp-data:/app/data --name web3 nginx
echo "From Host" > ~/myapp-data/host-file.txt
docker exec web3 cat /app/data/host-file.txt

```

```

aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ mkdir ~/myapp-data
docker run -d -v ~/myapp-data:/app/data --name web3 nginx
echo "From Host" > ~/myapp-data/host-file.txt
docker exec web3 cat /app/data/host-file.txt
0ef975ef0bf4f9834c394c680df077976e8bc07f0e8d848894ffe738e0b0bae4
From Host

```

## Lab 3: Practical Examples

### MySQL Persistent Storage

```
docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
Unable to find image 'mysql:8.0' locally
8.0: Pulling from library/mysql
60c3210ab1f6: Pull complete
10ee8ef3b9fc: Pull complete
cd8097e7c26b: Pull complete
0940ecf5daec: Pull complete
c3b8cfb10b41: Pull complete
440208b3f29f: Pull complete
5915b0e5580c: Pull complete
6375ec02106f: Pull complete
771773a1fe7a: Pull complete
1378718ee973: Pull complete
d4c7048d1cf1: Pull complete
de0490608ca2: Download complete
2f1dcce71f1f: Download complete
Digest: sha256:64756cc92f707eb504496d774353990bcb0f6999ddf598b6ad188f2da66bd000
Status: Downloaded newer image for mysql:8.0
aa5cc1cbd019cce39b4c1f0868acdb9e1d2541fcc6cd0b6967c7ab2d3cd97186
```

```
docker stop mysql-db
docker rm mysql-db
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker stop mysql-db
docker rm mysql-db
mysql-db
mysql-db
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
--name new-mysql \
-v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0
7833f4de09d84fa230a8ca16b4fdaf1102152aaadf5a7d021522a620d4327ca6
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

## NGINX Config Mount

```
mkdir ~/nginx-config
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ mkdir ~/nginx-config
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
echo 'server {
    listen 80;
    location / {
        return 200 "Hello from mounted config!";
    }
}' > ~/nginx-config/nginx.conf
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ echo 'server {
    listen 80;
    location / {
        return 200 "Hello from mounted config!";
    }
}' > ~/nginx-config/nginx.conf
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
docker run -d \
--name nginx-custom \
-p 8080:80 \
-v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
nginx
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
--name nginx-custom \
-p 8080:80 \
-v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
nginx
e5f71609dafe98d4f078773fee69f3a22848867038bc8d4a9559e819b70ebf45
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
curl http://localhost:8080
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ curl http://localhost:8080
Hello from mounted config!aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

## Lab 4: Volume Commands

```
docker volume ls
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume ls
DRIVER      VOLUME NAME
local       assignment1_pgdata
local       caaec63d137f1153e400de0ced5c7fee4dd3ac58e966e75fa095a9a07720630b
local       containerassgn_pgdata
local       containerzatiassgn_postgres_data
local       dockerassignment_postgres_data
local       minikube
local       mydata
local       mysql-data
local       myvolume
local       myvolumej
local       project_pg_data
local       projectassignment_pgdata
```

```
docker volume create app-volume
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume create app-volume
app-volume
```

```
docker volume inspect app-volume
```

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume inspect app-volume
[
  {
    "CreatedAt": "2026-04-06T18:33:21Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/app-volume/_data",
    "Name": "app-volume",
    "Options": null,
    "Scope": "local"
  }
]
```

docker volume prune

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume prune
WARNING! This will remove anonymous local volumes not used by at least one container
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume rm volume-name
Error response from daemon: volume volume-name not found
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume rm app-volume
```

docker volume **rm** volume-name

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker volume rm app-volume
app-volume
```

docker **cp** local-file.txt container-name:/path/in/volume

```
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker cp local-file.txt test-container:/data
Successfully copied 2.05kB to test-container:/data
aaroHi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$
```

## Part 2: Environment Variables

### Setting Variables

```
docker run -d \
  --name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
  my-node-app
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ docker run -d \
--name app1 \
-e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
-e DEBUG="true" \
ubuntu sleep 300
339cbfdbaa127e66fa7de6b160d87b292e941ce2b208437b452c20b4fbb9c330
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

## Using .env File

```
echo "DATABASE_HOST=localhost" > .env
echo "API_KEY=secret123" >> .env
```

```
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ echo "DATABASE_HOST=localhost" > .env
echo "API_KEY=secret123" >> .env
aarohi26@LAPTOP-4MMN5FCL:/mnt/c/Users/ASUS$ |
```

```
docker run -d --env-file .env my-app
```

## Dockerfile Example

```
ENV NODE_ENV=production
ENV PORT=3000
```

## Flask Example

```
import os
from flask import Flask

app = Flask(__name__)

@app.route('/config')
def config():
    return {
        "db": os.environ.get("DATABASE_HOST"),
        "debug": os.environ.get("DEBUG")
    }

app.run(host="0.0.0.0", port=5000)
```

## Test Variables

```
docker run -d \  
  --name flask-app \  
  -p 5000:5000 \  
  -e DEBUG=true \  
  flask-app  
  
docker exec flask-app env  
curl http://localhost:5000/config
```

## Part 3: Docker Monitoring

```
docker stats  
docker stats --no-stream  
  
docker top container-name  
  
docker logs -f container-name  
  
docker inspect container-name  
  
docker events
```

## Simple Monitoring Script

```
#!/bin/bash  
docker ps  
docker stats --no-stream  
docker system df
```

## Part 4: Docker Networks

## List Networks

```
docker network ls
```

## Bridge Network

```
docker network create my-network
```

```
docker run -d --name web1 --network my-network nginx
```

```
docker run -d --name web2 --network my-network nginx
```

```
docker exec web1 curl http://web2
```

## Host Network

```
docker run -d --network host nginx
```

## None Network

```
docker run -d --network none alpine sleep 3600
```

## Network Commands

```
docker network create app-network
```

```
docker network connect app-network container-name
```

```
docker network disconnect app-network container-name
```

```
docker network rm app-network
```

## Multi-Container Example

```
docker network create app-network
```

```
docker run -d \  
  --name postgres-db \  
  --network app-network \  
  -e POSTGRES_PASSWORD=secret \  
  postgres:15
```

```
docker run -d \  
  --name web-app \  
  --network app-network \  
  -p 8080:3000 \  
  -e DATABASE_HOST="postgres-db" \  
  node-app
```

## Debugging

```
docker network inspect bridge  
docker exec container-name ping google.com  
docker port container-name
```

## Part 5: Real-World Example

```
docker network create myapp-network
```

```
docker run -d \  
  --name postgres \  
  --network myapp-network \  
  -e POSTGRES_PASSWORD=pass \  
  postgres:15
```

```
docker run -d \  
  --name redis \  
  --network myapp-network \  
  redis:7-alpine
```

```
docker run -d \  
  --name flask-app \  
  --network myapp-network \  
  -p 5000:5000 \  
  flask-app:latest
```

## Cheatsheet

```
docker volume create <name>
docker run -v <volume>:/path

docker run -e VAR=value

docker stats
docker logs -f <container>

docker network create <name>
```

## Practice

- Create DB with volume
- Multi-service setup
- Analyze logs
- Network isolation

## Cleanup

```
docker stop $(docker ps -aq)
docker rm $(docker ps -aq)
docker volume prune -f
docker network prune -f
docker image prune -f
```

## Key Takeaways

- Volumes store persistent data
- Env variables control config
- Monitoring helps debugging
- Networks connect containers
- Use .env for security